

1. Project Overview

The Forum Builder Fund DAO aims to provide a decentralized governance mechanism for community-building funds, specifically tailored for high-quality content communities and developer forums. A public budget is established on a fixed-cycle basis—funded by sponsorships, ad revenues, or membership fees—and securely stored within a DAO Treasury managed by smart contracts. This ensures that the funds are not directly controlled by a single forum administrator. The primary objective is to empower authentic community contributors (whitelisted members) to collaboratively determine the allocation of limited resources through proposals and voting, with the smart contract automatically executing the disbursements based on the governance results.

The Minimum Viable Product (MVP) of this project intentionally omits complex reputation systems and tokenomics to focus strictly on the core functionalities: **proposal submission, budget-constrained ranked voting, and automated fund execution via smart contracts.**

2. System Architecture

This project adopts a **"No Frontend, Fat Backend, Thin Contract"** architectural pattern.

1. Client Layer:

- The system does not include a web frontend interface. Instead, it exposes RESTful APIs or a Command Line Interface (CLI) via the Java backend, allowing users (whitelisted members and administrators) to submit proposals and cast votes.

2. Service Layer (Core Backend):

- The core controller is built using **Java (Spring Boot / Core Java)**.
- It handles complex off-chain business logic, including the dynamic calculation of whitelisted members' active status, the execution of the 3-2-1 ranked voting algorithm, and initial formal reviews/interceptions of proposals.
- The **Web3j** library is utilized to facilitate communication between the Java backend and the underlying blockchain network.

3. Blockchain Layer (Smart Contracts):

- The DAO Treasury smart contract is deployed on a **Hardhat** local test network.
- It acts solely as the absolute security baseline for funds: locking proposal deposits, verifying disbursement instructions sent from the backend, and automatically transferring funds to the winning proposals' designated wallets.

3. Business Logic and Workflows

The system operates on a fixed-cycle basis (Funding Round, e.g., every 30 days). The business workflow for each cycle is divided into three main phases:

- **Phase 1: Proposal Submission**

- The system verifies the active whitelisted status of the user.
- The user submits the proposal details (including the recipient address and requested amount) and locks a refundable deposit of 50 USDC via the smart contract.
- **Phase 2: Review & Voting**
 - The backend system conducts a formal review of the proposals (e.g., checking if the requested amount exceeds the round's limit). Non-compliant proposals are immediately rejected.
 - Once the voting period commences, active members cast their votes using a 3-2-1 scoring system. During this period, voting data is encrypted/temporarily stored in the backend and is not disclosed in real-time.
- **Phase 3: Settlement & Execution**
 - Upon the conclusion of the voting period, the backend verifies the Quorum (which must reach at least 40% of total active members and consist of no fewer than 8 voters).
 - Proposals that meet the approval threshold are sorted in descending order based on their total scores. The final list of approved proposals is determined by the fixed budget constraint of the current round.
 - The backend utilizes Web3j to call the `executeTreasury` function on the smart contract, which automatically batches the disbursements to the winning proposals and refunds the deposits to compliant proposers.

4. Design Justifications (Explain why you design it like that)

Our design choices for the core mechanisms are justified as follows:

- **Why Blockchain with a Whitelist?**
 - **Sybil Attack Prevention:** Forum governance is highly susceptible to bots and malicious actors. A whitelist maintained by a multi-sig committee ensures that only genuine community contributors participate.
 - **Trustless Treasury Execution:** Even within a trusted community, relying on a single administrator to manage public funds introduces risks of corruption or mismanagement. The smart contract acts as an impartial "automated cashier." Once the voting results are finalized, no individual can block the fund disbursement, achieving true trustless fund custody.
- **Why a Ranking & Budget-Constrained Mechanism?**
 - The community incentive budget is fixed for each funding round. Under limited resources, governance is essentially a competition among multiple high-quality proposals. An absolute evaluation system (e.g., funding any proposal that passes a minimum threshold) would lead to early submissions draining the treasury. The ranked scoring mechanism ensures maximum utility by prioritizing the most widely supported proposals within the budget constraint.
- **Why a "Fat Backend, Thin Contract" Architecture?**

- **Gas Optimization & Efficiency:** Performing complex operations on-chain—such as floating-point calculations for the 40% Quorum or iterating through large arrays for 3-2-1 sorting—is highly inefficient and gas-expensive. Offloading these heavy statistical computations to the Java backend while reserving the blockchain strictly for fund transfers is a highly pragmatic engineering approach.
- **Mitigating Collusion (Delayed Reveal):** By temporarily storing voting records in the backend and only publishing them to the chain/public after the voting period ends, the system effectively prevents bandwagon effects, vote-buying, and collusion during the voting phase.

5. Code & Technical Implementation

- **Backend Implementation:** The core logic of this project is implemented in Java, which handles the active member management module and the vote-tallying algorithm module.
- **Blockchain Integration:** A local Ethereum test network is established using Hardhat, where a streamlined DAO Treasury Solidity contract is deployed. The Java Web3j library is utilized to facilitate backend reading of on-chain states and triggering of transactions.
- **Source Code (GitHub Repository):** [Please insert your GitHub repository link here]

6. Survey on Peer Evaluation

- **Peer Evaluation Form:** [Please insert the link or reference to your peer evaluation document here]